NOVA SCHOOL OF SCIENCE & TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

MEI - Master in Computer Science and Engineering

Internet Applications Design and Implementation

Week 3 - Software architecture

Information

The dates of Test 1 and Test 2 were updated as instructed by the Master's program coordination.

Test 1	30 Apr
Test 2	23 May

Both the site (github) and the slides from week 1 were updated.

This is also in CLIP.

© Pedro Alves 2026

Software architecture

“The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.”

(in Software Architectures in Practice)

© Pedro Alves 2026

Software architecture

Three kinds of structures:

- Module structures
  code and data units (implementation)
- Component-and-connector structures
  elements that have runtime behaviour (components) and interactions (connectors)
- Allocation structures
  mapping between software structures and system's non-software structures (deployment and/or organizational)

© Pedro Alves 2026

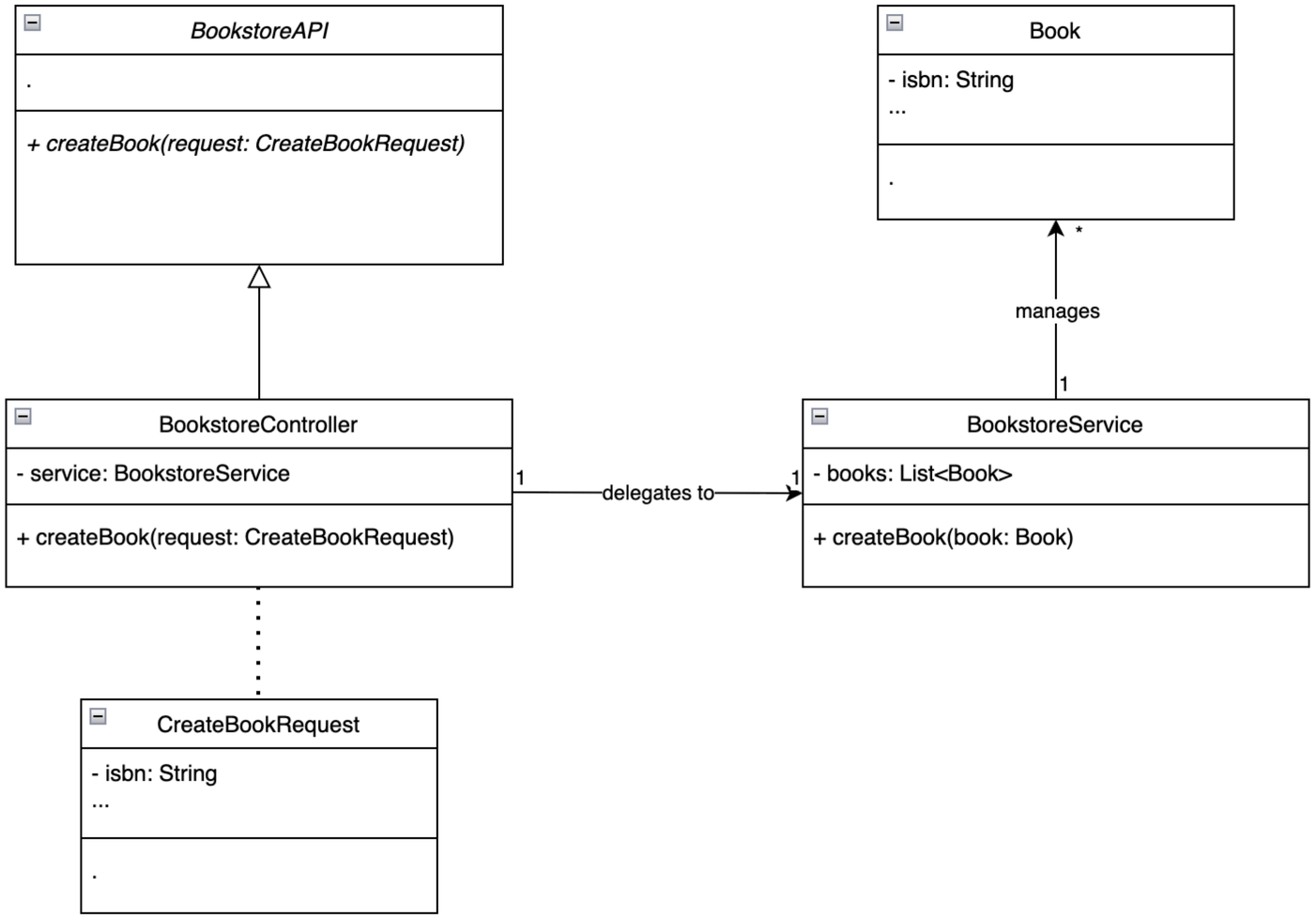
Software architecture

Implementation

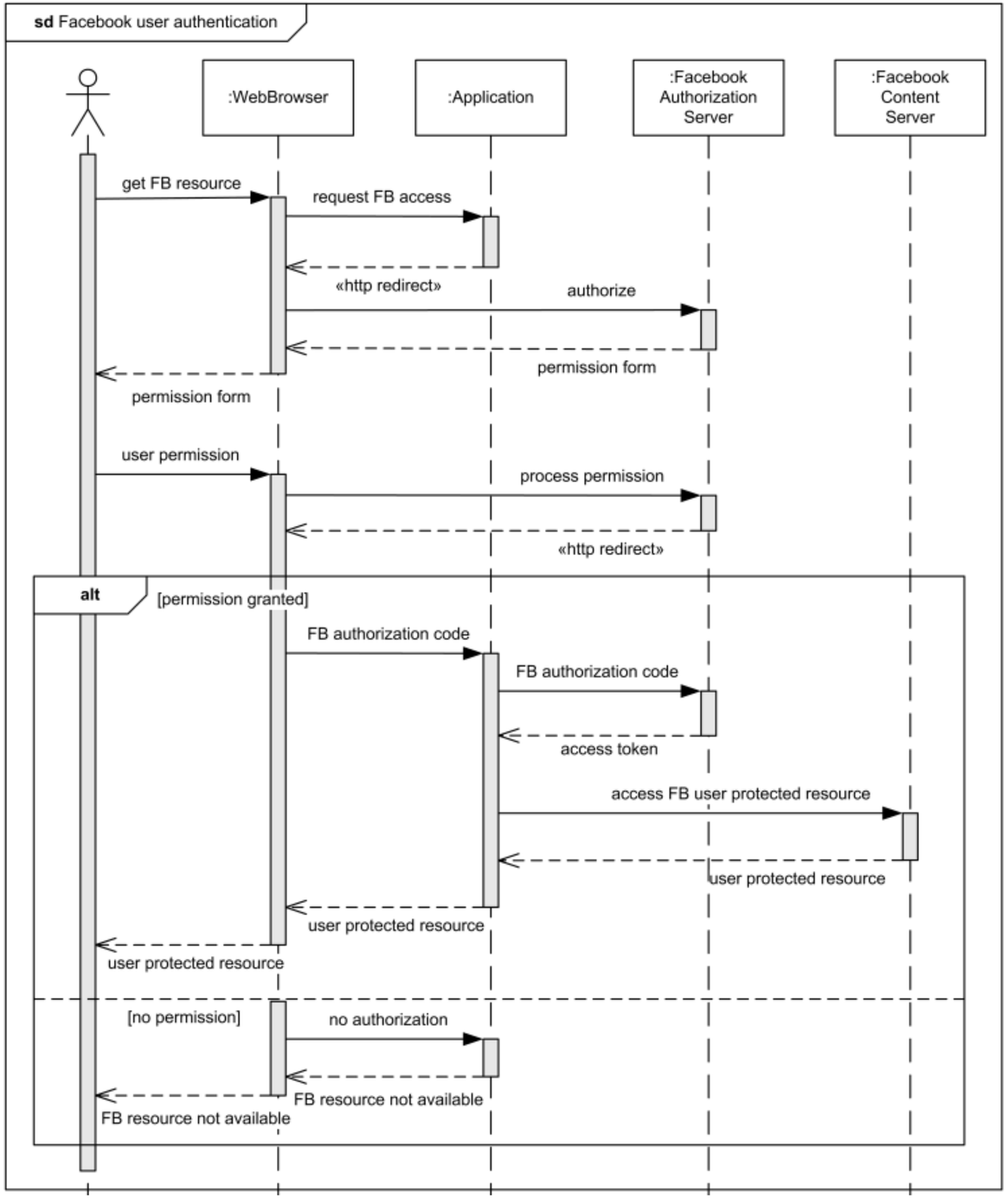
Examples:

UML Class Diagram

E/R Diagram



© Pedro Alves 2026



Software architecture

Runtime

Examples:

UML Sequence Diagram

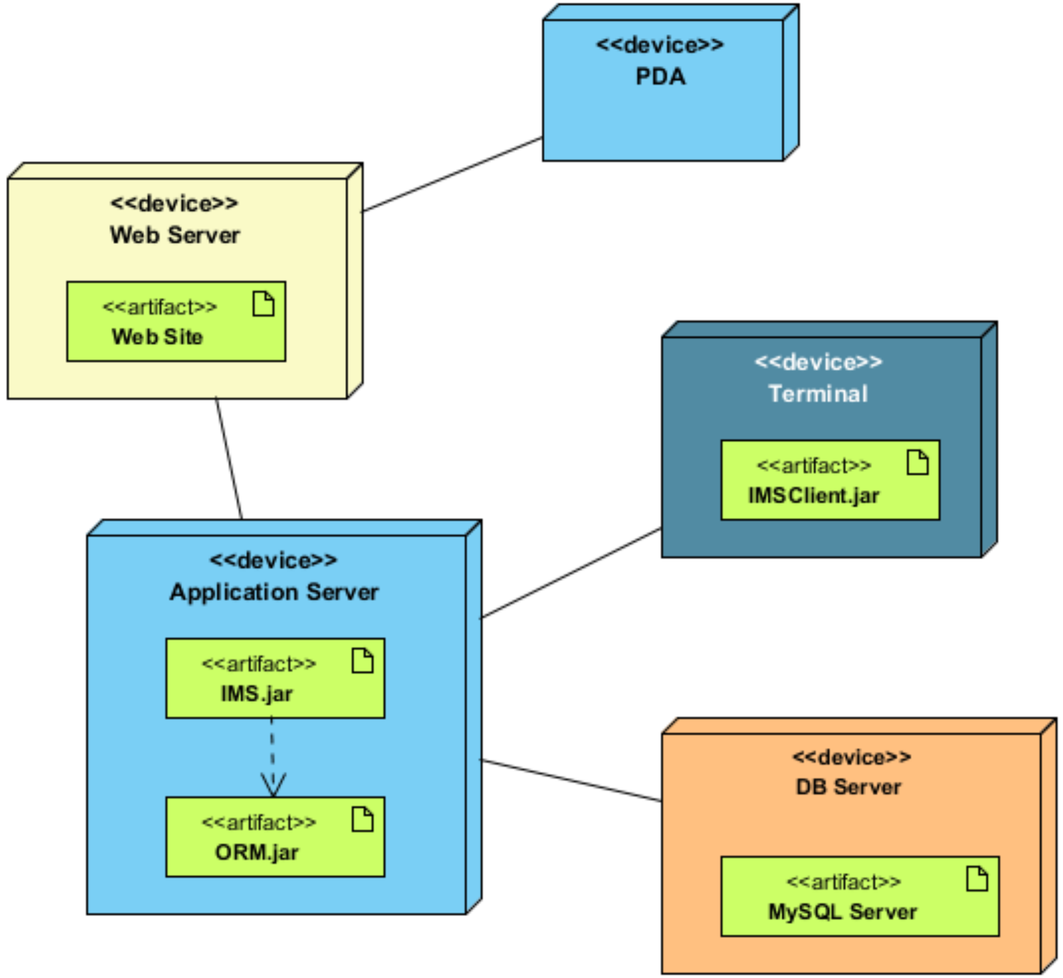
UML Activity Diagram

© Pedro Alves 2026

Deployment

Examples:

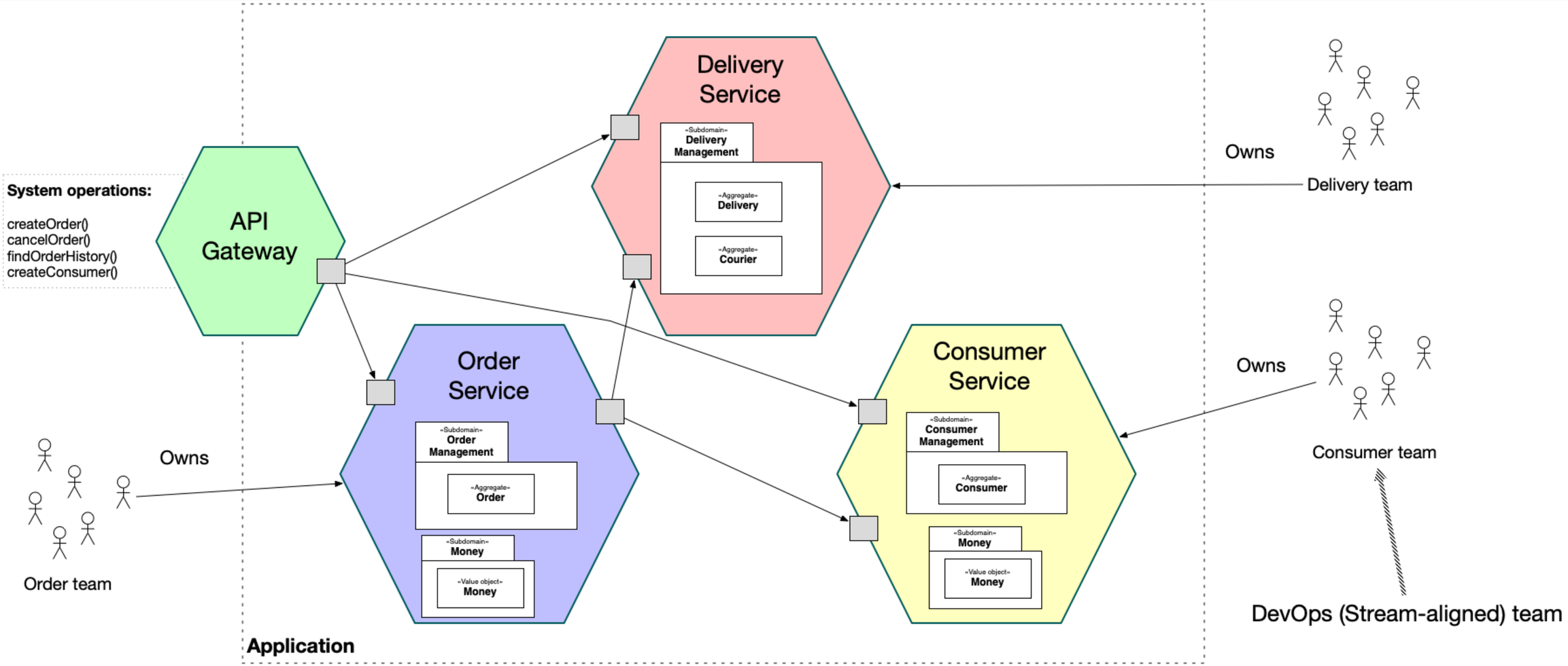
UML Deployment Diagram



© Pedro Alves 2026

Software architecture

Organizational



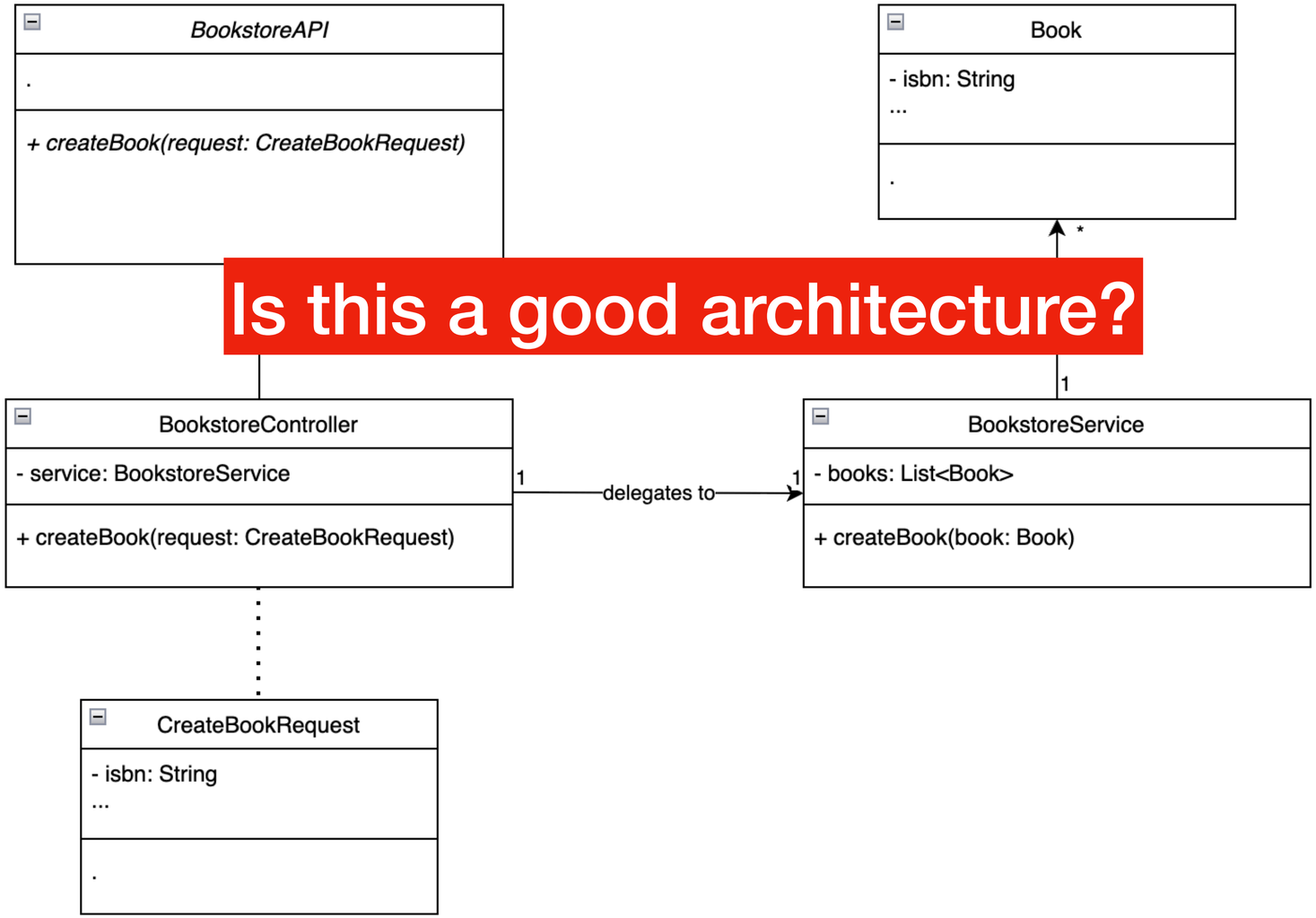
© Pedro Alves 2026

Software architecture

What is a good architecture?

© Pedro Alves 2026

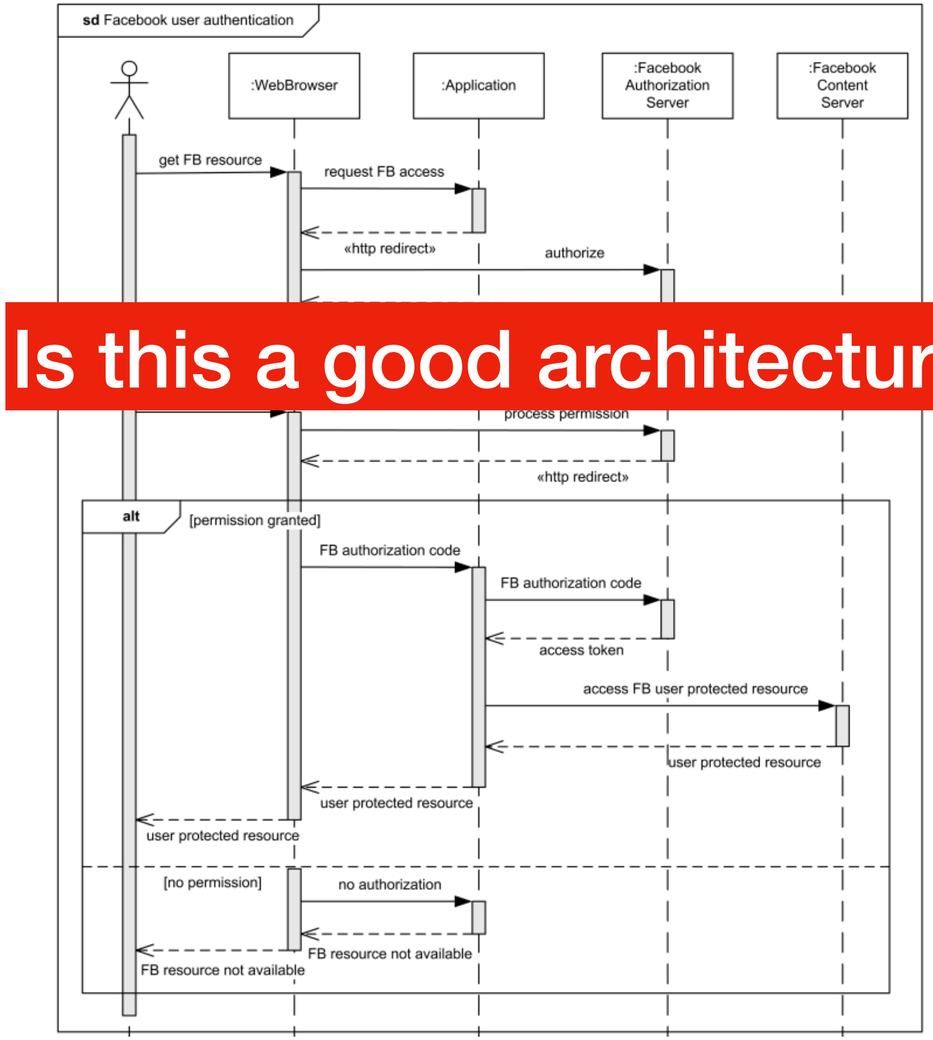
Software architecture



Is this a good architecture?

© Pedro Alves 2026

Software architecture



Is this a good architecture?

© Pedro Alves 2026

Software architecture

What is a good architecture?

- Productive development
- Easy evolution/maintenance
- Trustworthiness (correct, secure, …)
- Performance

© Pedro Alves 2026

Software architecture

- Implementation
  - module responsibilities, dependencies, type of relation (e.g., inheritance vs composition)
- Runtime
  - modules interactions, shared data stores, data flow, security requirements, performance
- Deployment
  - hardware/cloud infrastructure
- Organizational
  - module ownership

© Pedro Alves 2026

Software architecture

- Implementation productive development
  - module responsibilities, dependencies, type of relation (e.g., inheritance vs composition)
- Runtime
  - modules interactions, shared data stores, data flow, security requirements, performance
- Deployment
  - hardware/cloud infrastructure
- Organizational productive development
  - module ownership

© Pedro Alves 2026

Software architecture

- Implementation productive development easy evolution
  - module responsibilities, dependencies, type of relation (e.g., inheritance vs composition)
- Runtime
  - modules interactions, shared data stores, data flow, security requirements, performance
- Deployment easy maintenance
  - hardware/cloud infrastructure
- Organizational productive development
  - module ownership

© Pedro Alves 2026

Software architecture

- Implementation productive development easy evolution
  - module responsibilities, dependencies, type of relation (e.g., inheritance vs composition)
- Runtime secure
  - modules interactions, shared data stores, data flow, security requirements, performance
- Deployment easy maintenance secure
  - hardware/cloud infrastructure
- Organizational productive development
  - module ownership

© Pedro Alves 2026

Software architecture

- Implementation productive development easy evolution
  - module responsibilities, dependencies, type of relation (e.g., inheritance vs composition)
- Runtime secure performance
  - modules interactions, shared data stores, data flow, security requirements, performance
- Deployment easy maintenance secure performance
  - hardware/cloud infrastructure
- Organizational productive development
  - module ownership

© Pedro Alves 2026

Software architecture

Implementation

Define a system as a set of modules and their dependencies

module

A logical unit of software that groups related components behind a defined abstraction boundary

dependency

A relationship where one module relies on another to perform its responsibilities

© Pedro Alves 2026

Modularization & Dependencies

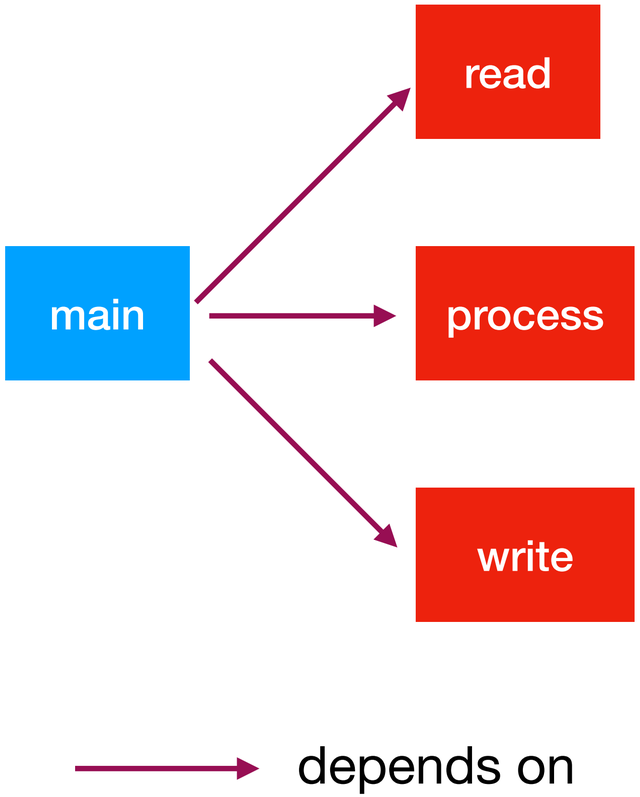
```
fun main() {
    println("Hello World!")
}
```

- Single unit
- No dependencies
- No modularization needed
- Everything in one place

© Pedro Alves 2026

Modularization & Dependencies

```
fun main() {
    val content = read()
    val result = process(content)
    write(result)
}
```



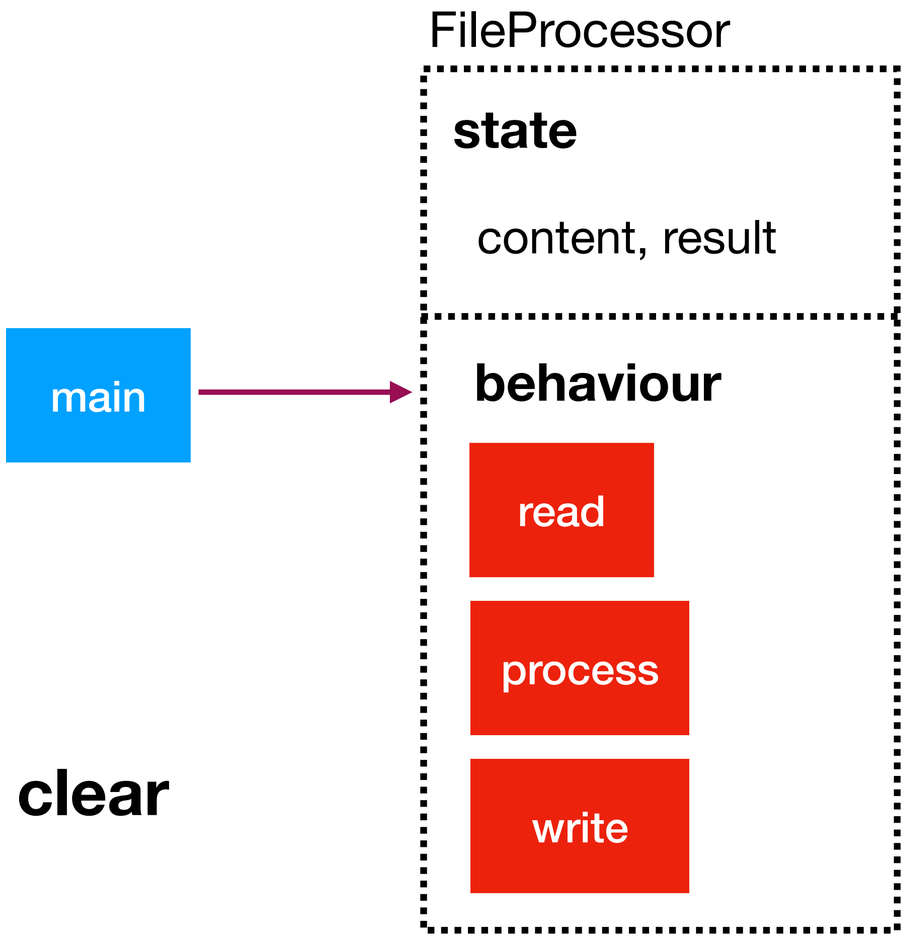
- main <u>depends</u> on read, process and write
- benefits: reuse, testability, readability

functions = abstraction boundaries

© Pedro Alves 2026

Modularization & Dependencies

```
fun main() {
    val fileProcessor = FileProcessor()
    fileProcessor.read()
    fileProcessor.process()
    fileProcessor.write()
}
```



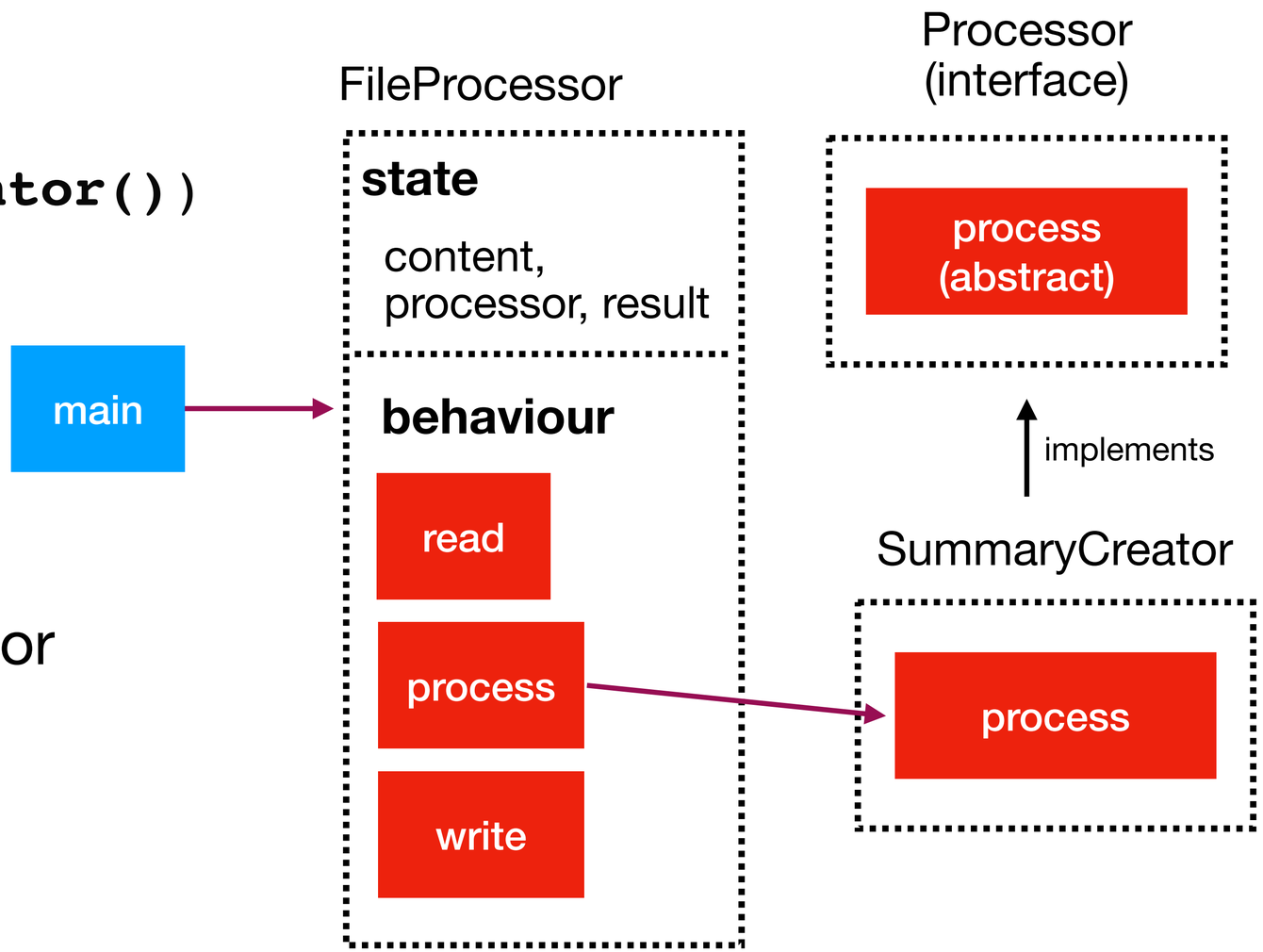
- main <u>depends</u> on FileProcessor
- benefits: increased reuse, testability, readability, **clear ownership of state, separation of concerns**

classes = abstraction boundaries

© Pedro Alves 2026

Modularization & Dependencies

```
fun main() {
    val fileProcessor =
      FileProcessor(SummaryCreator())
    fileProcessor.read()
    fileProcessor.process()
    fileProcessor.write()
}
```

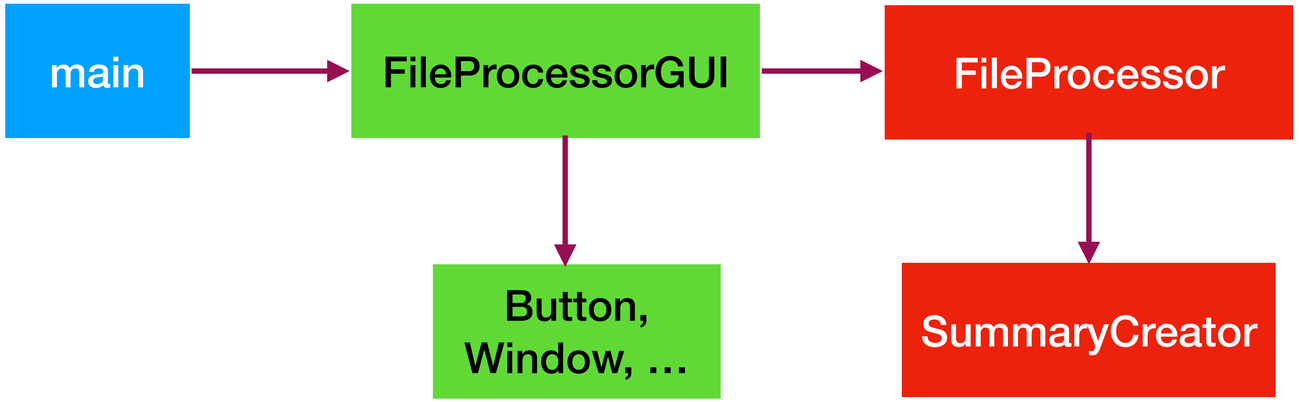


- main <u>depends</u> on FileProcessor which depends on SummaryCreator (**dependency injection**)
- even more reusability!

© Pedro Alves 2026

Modularization & Dependencies

```
fun main() {
    val fileProcessorGUI =
        FileProcessorGUI(
            FileProcessor(
                SummaryCreator()
            )
        )
    fileProcessorGUI.show()
}
```



- main depends on FileProcessorGUI which depends on FileProcessor

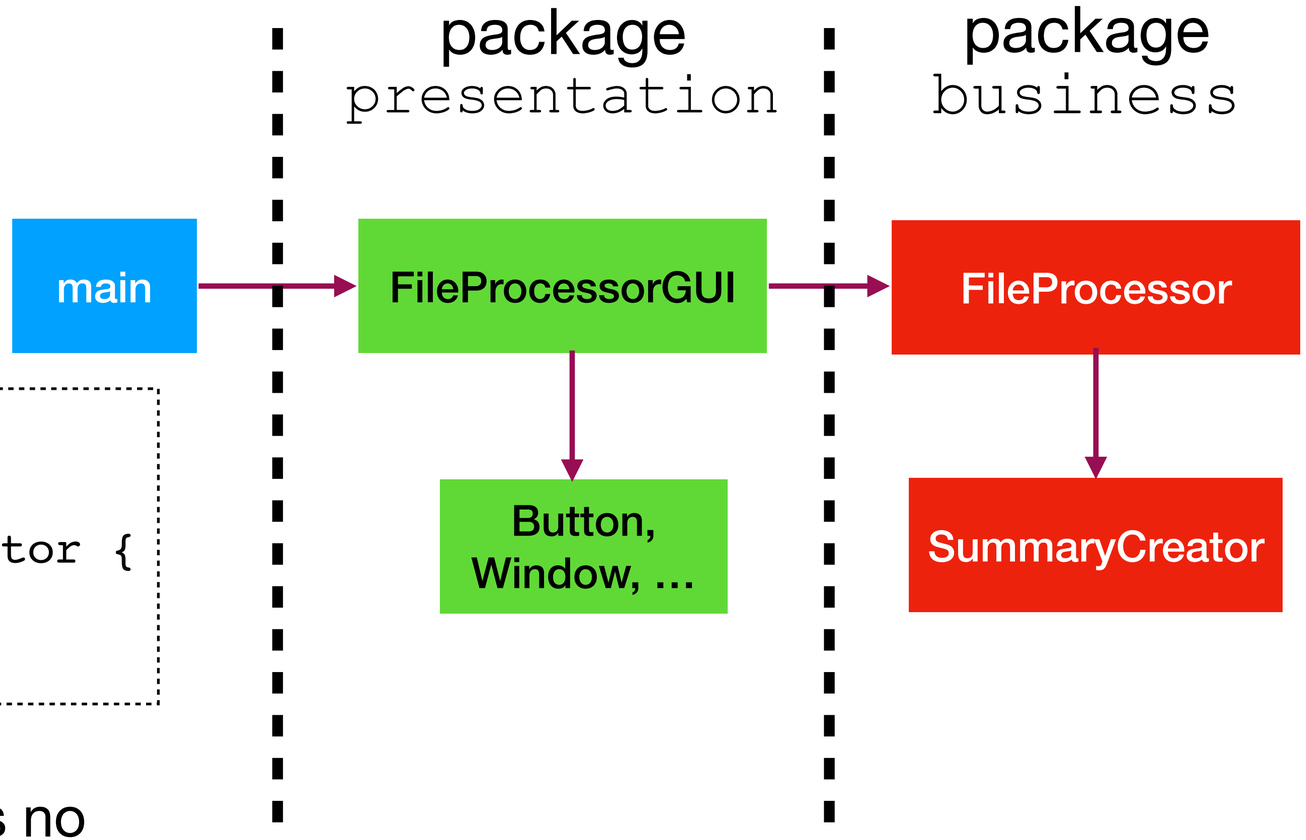
© Pedro Alves 2026

Modularization & Dependencies

```
package presentation

class FileProcessorGUI {
    ...
}
```

```
package business

class FileProcessor {
    ...
}
```

```
package business

class SummaryCreator {
    ...
}
```



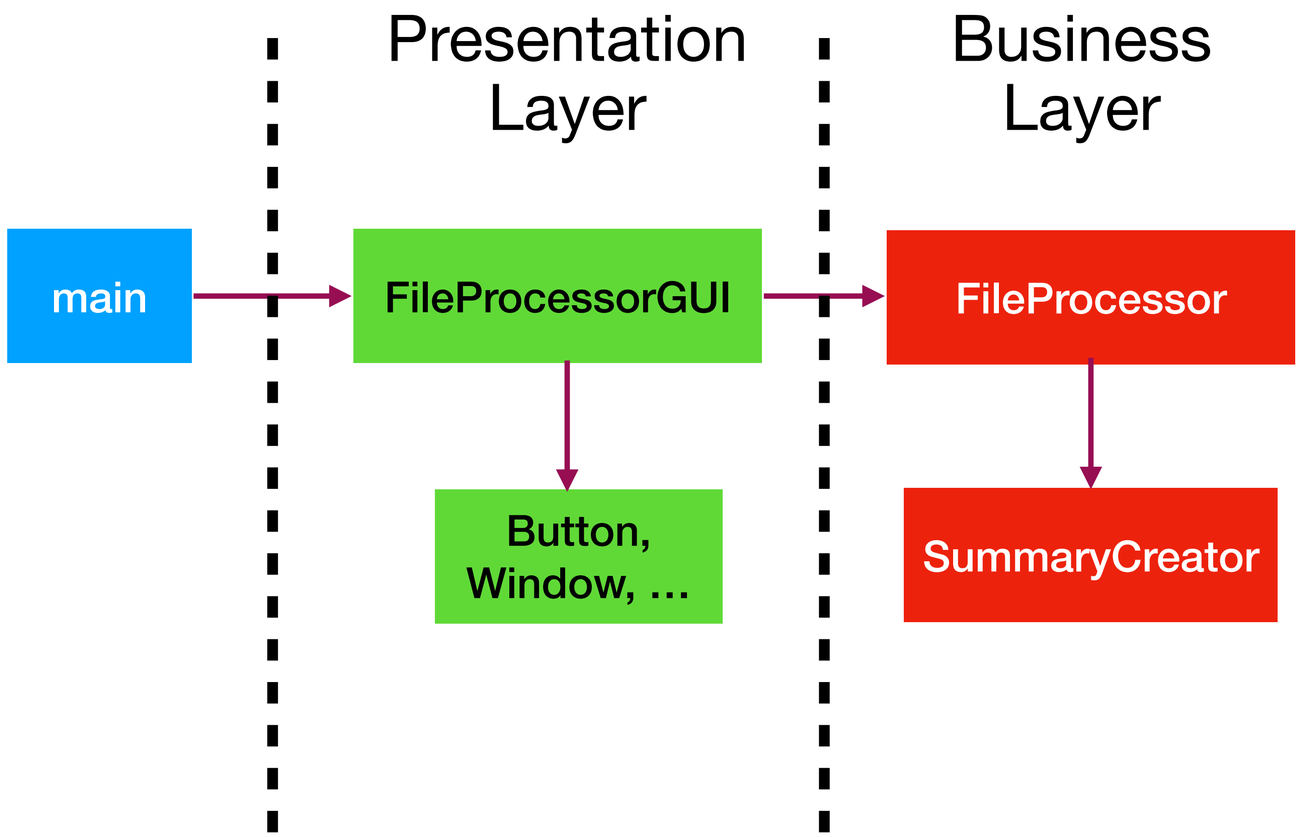
packages are arbitrary sets of classes - there is no boundary enforcement (unless you use Spring Modulith)

packages = abstraction boundaries

© Pedro Alves 2026

Modularization & Dependencies

```
fun main() {
    val fileProcessorGUI =
        FileProcessorGUI(
            FileProcessor(
                SummaryCreator()
            )
        )
    fileProcessorGUI.show()
}
```



- main depends on FileProcessorGUI which depends on FileProcessor

layers = abstraction boundaries

© Pedro Alves 2026

Detour - Layers

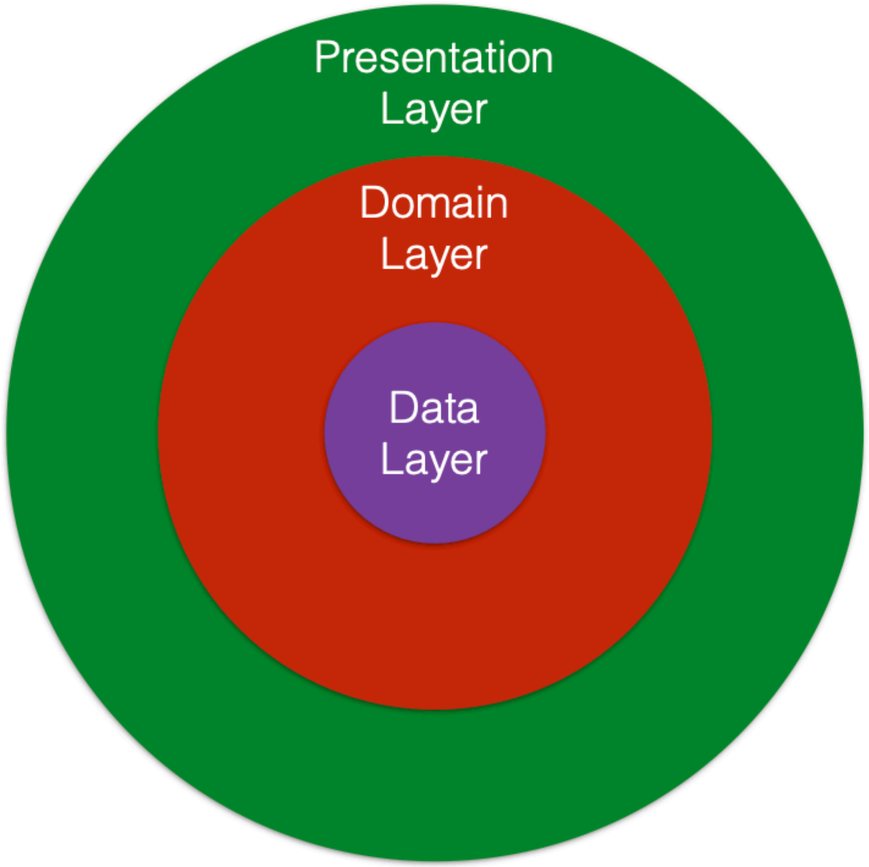
Layered pattern

- Relation between layers is unidirectional
- Layers interact according to a strict ordering relation
- Layers interact only with the next layer (can’t jump layers)

© Pedro Alves 2026

Detour - Layers

Data-centric applications



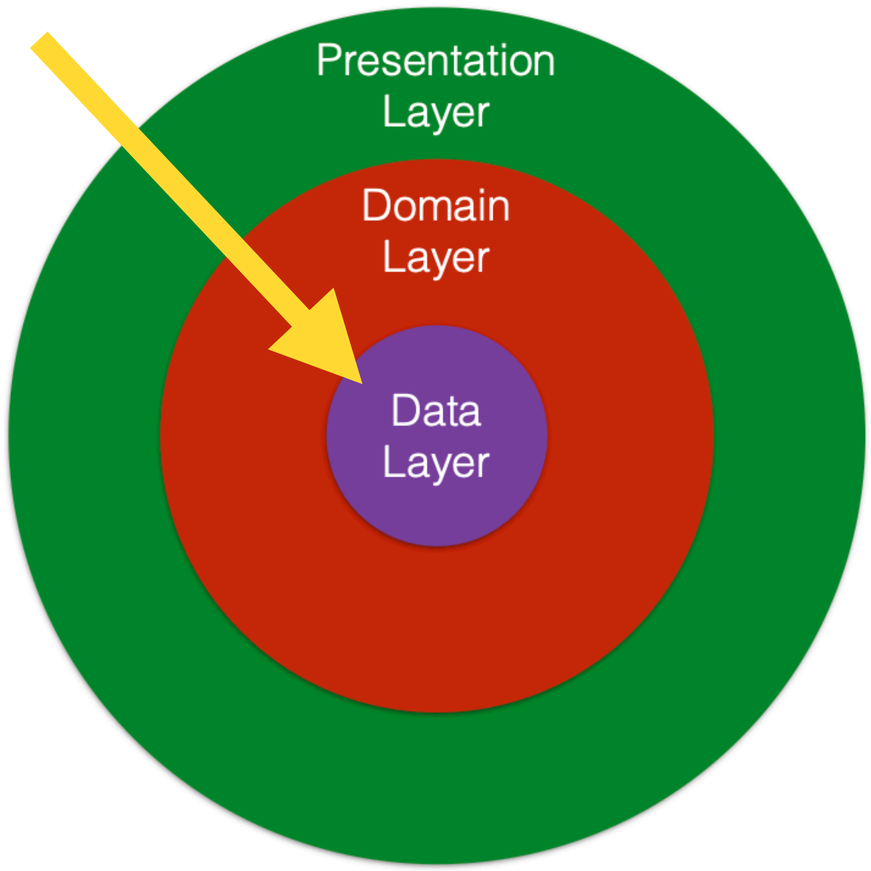
© Pedro Alves 2026

Detour - Layers

Interaction starts with the outer layer (Presentation Layer)

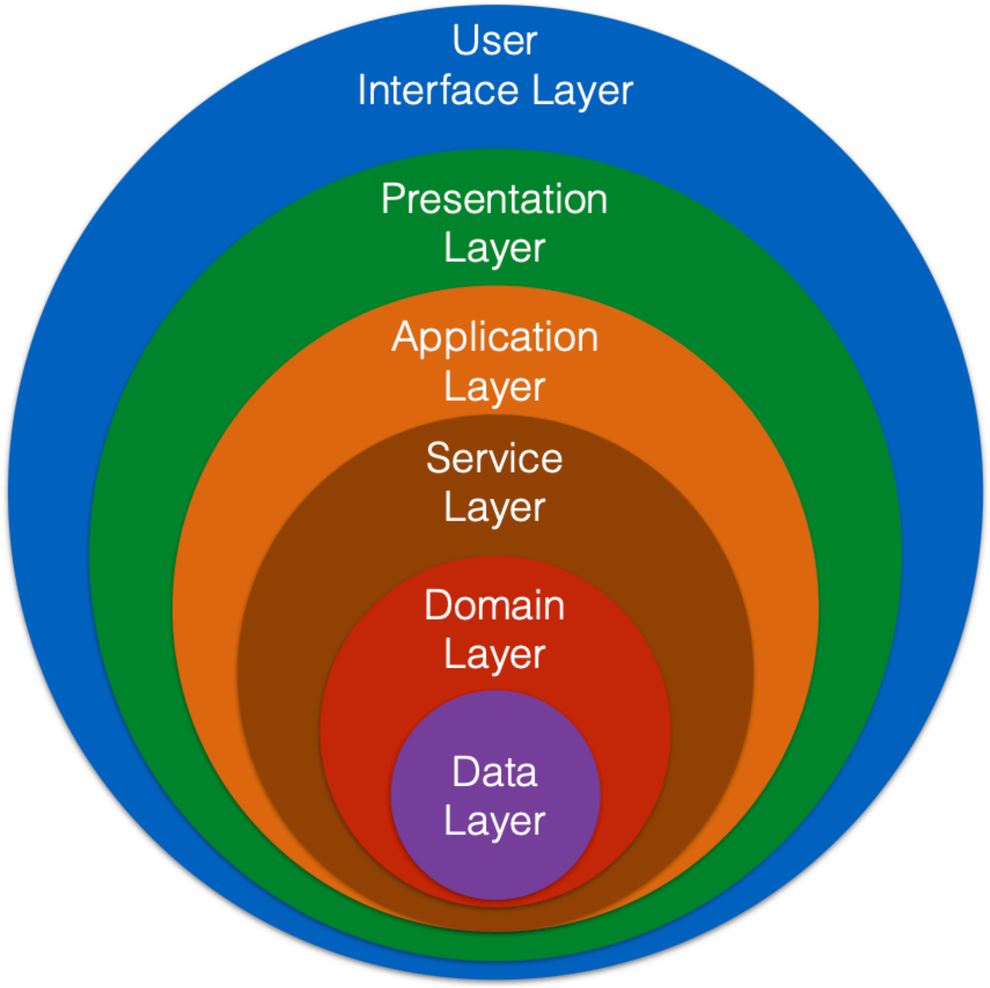
Presentation Layer must talk with the Domain Layer, can’t talk directly with Data Layer

Domain Layer must talk with the Data Layer



© Pedro Alves 2026

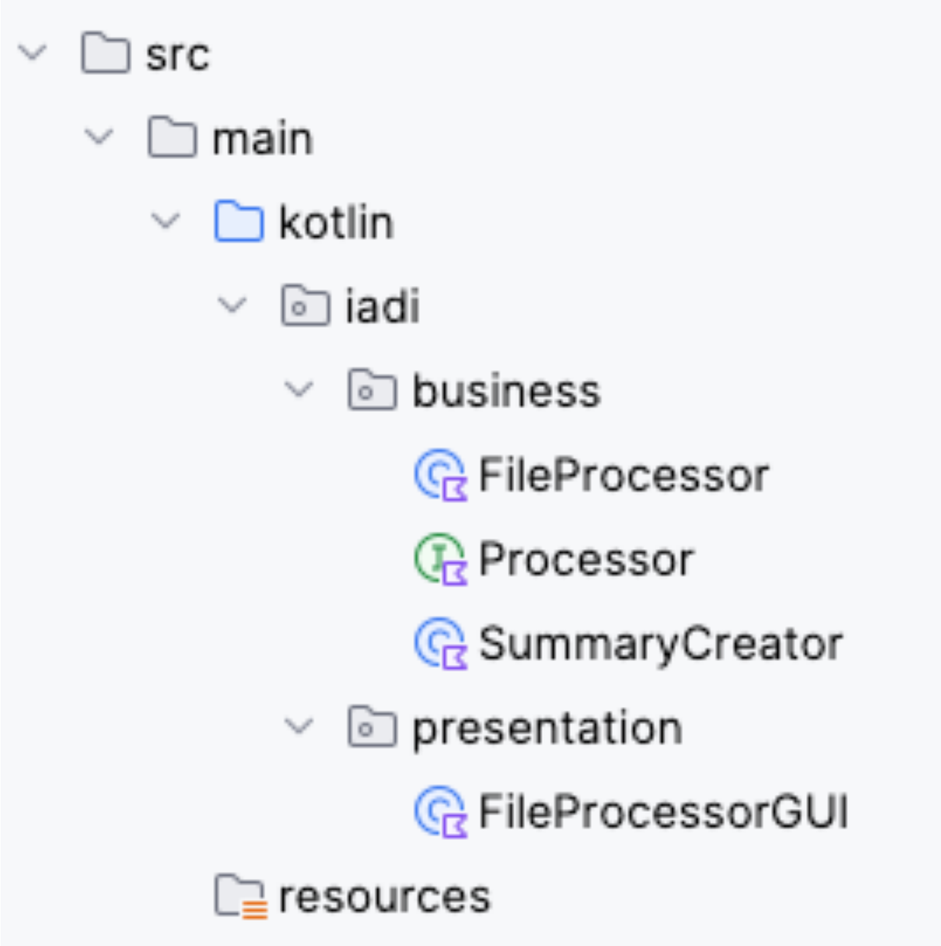
Detour - Layers



https://dzone.com/articles/layered-architecture-is-good

© Pedro Alves 2026

Modularization & Dependencies



Single code base

- benefits: atomic changes (refactoring, etc…), no versioning problems, simpler debugging, faster development
- problems: weak boundaries, difficult to scale across teams, no incremental deployment, long build times, increased merge conflicts

© Pedro Alves 2026

Detour - Maven multi-module

```
my-project/
├── pom.xml (Parent / Aggregator POM)
│
├── common/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   └── com/example/common/
│       │   └── resources/
│
├── service/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   └── com/example/service/
│       │   └── resources/
│
└── web/
    ├── pom.xml
    └── src/
        ├── main/
        │   ├── java/
        │   │   └── com/example/web/
        │   ├── resources/
        │   └── webapp/
```

Maven Project with:

- Parent
  - Usually has only a POM
  - Defines shared dependencies
  - Lists modules
  - Builds everything together
- Subprojects (modules)
  - Each one has its own POM
  - References the parent
  - Defines specific dependencies
  - Can depend on other modules

© Pedro Alves 2026

Detour - Maven multi-module

```
my-project/
├── pom.xml (Parent / Aggregator POM)
│
├── common/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   └── com/example/common/
│       │   └── resources/
│
├── service/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   ├── java/
│       │   │   └── com/example/service/
│       │   └── resources/
│
└── web/
    ├── pom.xml
    └── src/
        ├── main/
        │   ├── java/
        │   │   └── com/example/web/
        │   ├── resources/
        │   └── webapp/
```

sub-module

sub-module

sub-module

maven sub-modules = abstraction boundaries

© Pedro Alves 2026

Detour - Maven multi-module

/pom.xml

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>my-project</artifactId>
  <version>1.0.0</version>
  <packaging>pom</packaging>

  <modules>
    <module>common</module>
    <module>service</module>
    <module>web</module>
  </modules>

  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-core</artifactId>
        <version>6.1.0</version>
      </dependency>
    </dependencies>
  </dependencyManagement>
</project>
```

list of modules

shared dependencies

/service/pom.xml

```
<project>
  <parent>
    <groupId>com.example</groupId>
    <artifactId>my-project</artifactId>
    <version>1.0.0</version>
  </parent>

  <artifactId>service</artifactId>

  <dependencies>
    <dependency>
      <groupId>com.example</groupId>
      <artifactId>common</artifactId>
      <version>${project.version}</version>
    </dependency>
  </dependencies>
</project>
```

reference to parent

depends on module “common”

© Pedro Alves 2026

Detour - Maven multi-module

In the project base folder:

```
mvn compile
```

Compiles all modules, manages dependencies between modules

```
mvn compile -pl service
```

Compiles just the service module

```
mvn compile -pl service -am
```

Compiles the service module and all its module dependencies

© Pedro Alves 2026

Detour - Maven multi-module

maven single-module

```
file-processor/
├── pom.xml
└── src/
    └── main/
        └── kotlin/
            └── iadi/
                ├── business/
                │   ├── FileProcessor.kt
                │   ├── Processor.kt
                │   └── SummaryCreator.kt
                └── presentation/
                    └── FileProcessorGUI.kt
```

maven multi-module

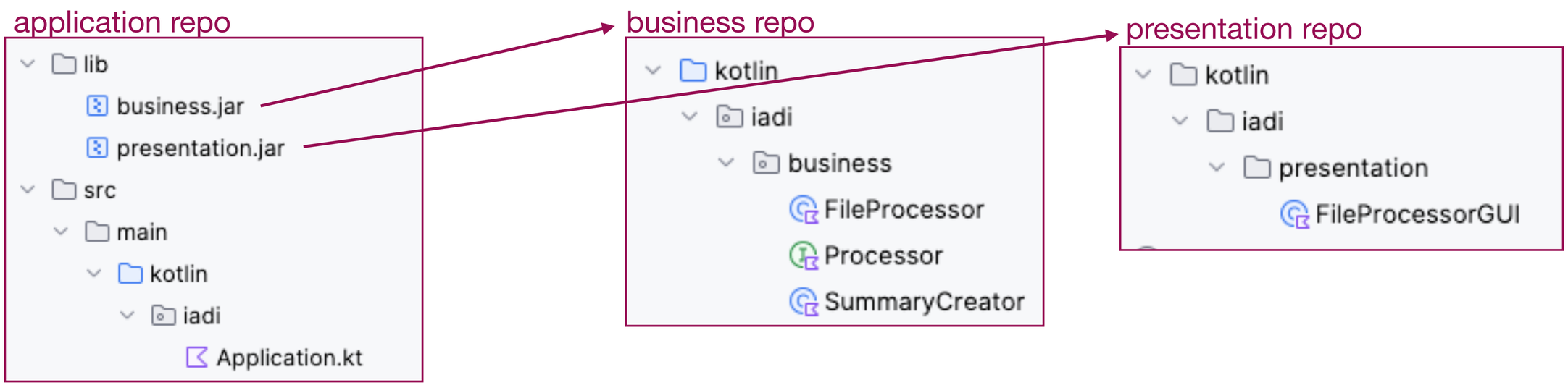
```
file-processor-parent/
├── pom.xml(parent / aggregator)
│
├── business/
│   ├── pom.xml
│   └── src/
│       ├── main/
│       │   └── kotlin/
│       │       └── iadi/
│       │           └── business/
│       │               ├── FileProcessor.kt
│       │               ├── Processor.kt
│       │               └── SummaryCreator.kt
│
└── presentation/
    ├── pom.xml
    └── src/
        ├── main/
        │   └── kotlin/
        │       └── iadi/
        │           └── presentation/
        │               └── FileProcessorGUI.kt
```

It's still a single code base

- benefits: atomic changes (refactoring, etc…), no versioning problems, simpler debugging, faster development
- **less** problems: **not so** weak boundaries, **not so** difficult to scale across teams, **allows** ~~no~~ incremental deployment, long build times, **slightly less** merge conflicts

© Pedro Alves 2026

Modularization & Dependencies



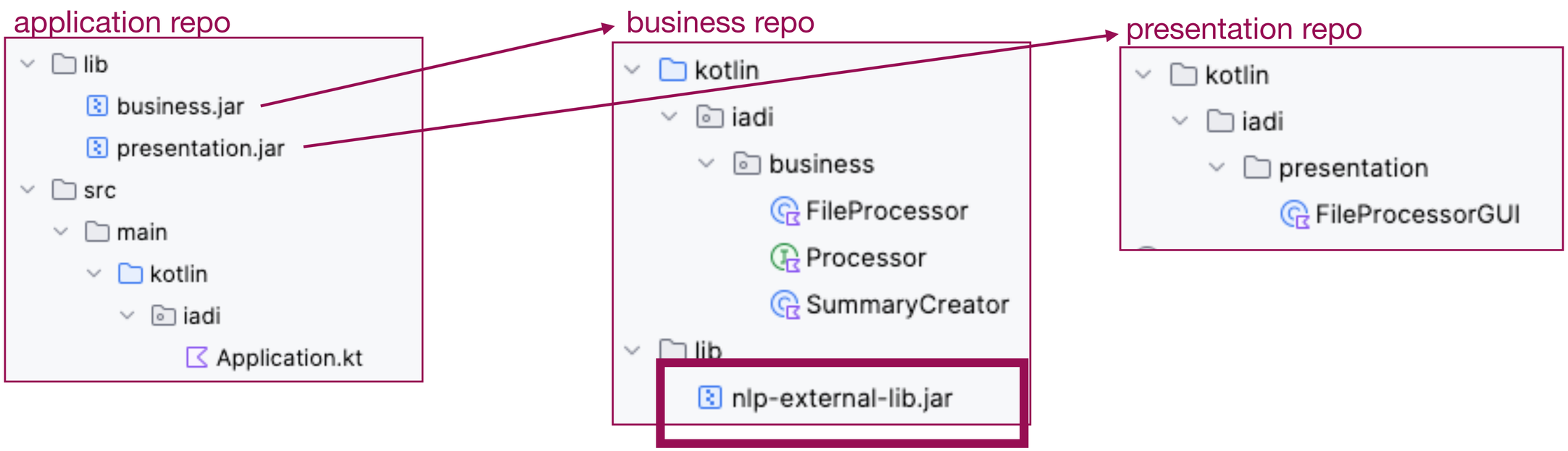
Different code bases => Different libraries (developed in-house)

- benefits: stronger module boundaries, independent teams, independent releases, clear ownership
- problems: versioning complexity, slower refactorings, dependency hell, over-modularization risks

libraries = abstraction boundaries

© Pedro Alves 2026

Modularization & Dependencies



Libraries (externally developed)

- benefits: “free” productivity, higher quality (often), standardization
- problems: dependency hell x 10, hidden complexity, harder debugging, lock-in

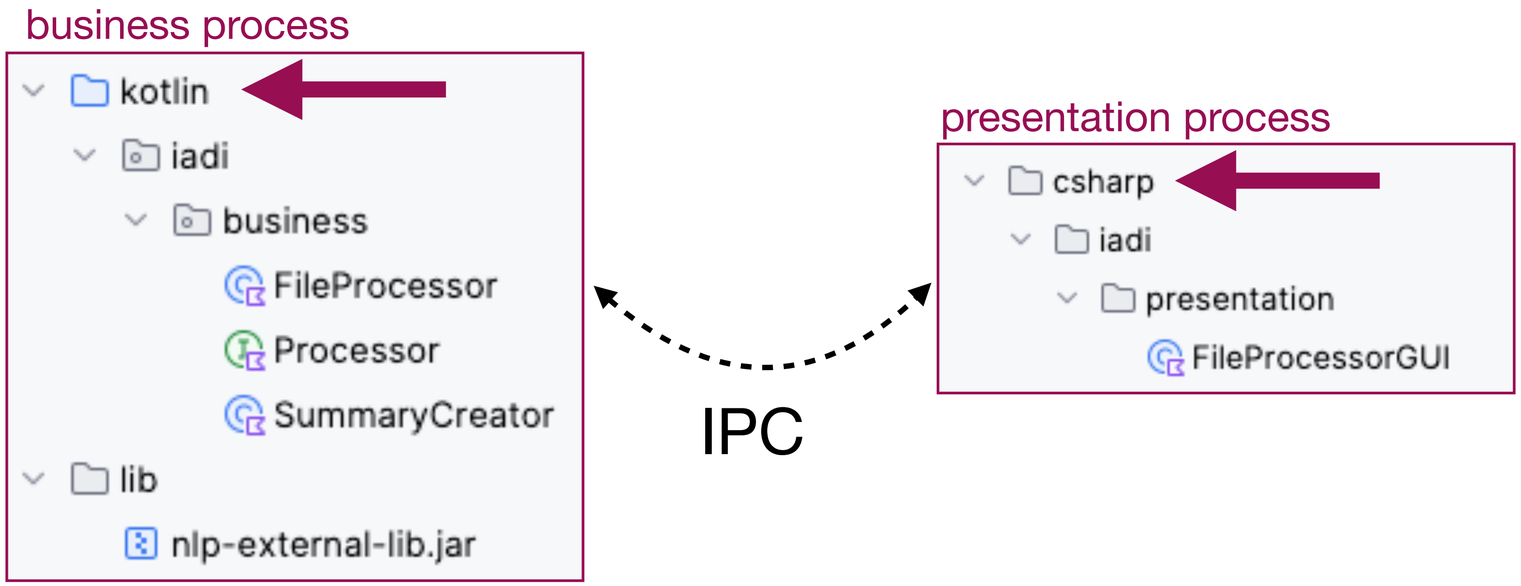
© Pedro Alves 2026

Detour - dependency management

Usually, dependencies are not managed manually (i.e., by putting JAR files inside a lib folder) but rather managed by a dependency management tool such as Maven, Gradle, pip, or npm.

© Pedro Alves 2026

Modularization & Dependencies



Separate OS processes (same machine)

Communication through IPC sockets, shared memory, common database

- benefits: language/runtime independence, fault isolation
- problems: serialization, deployment complexity

stronger decoupling through process boundaries

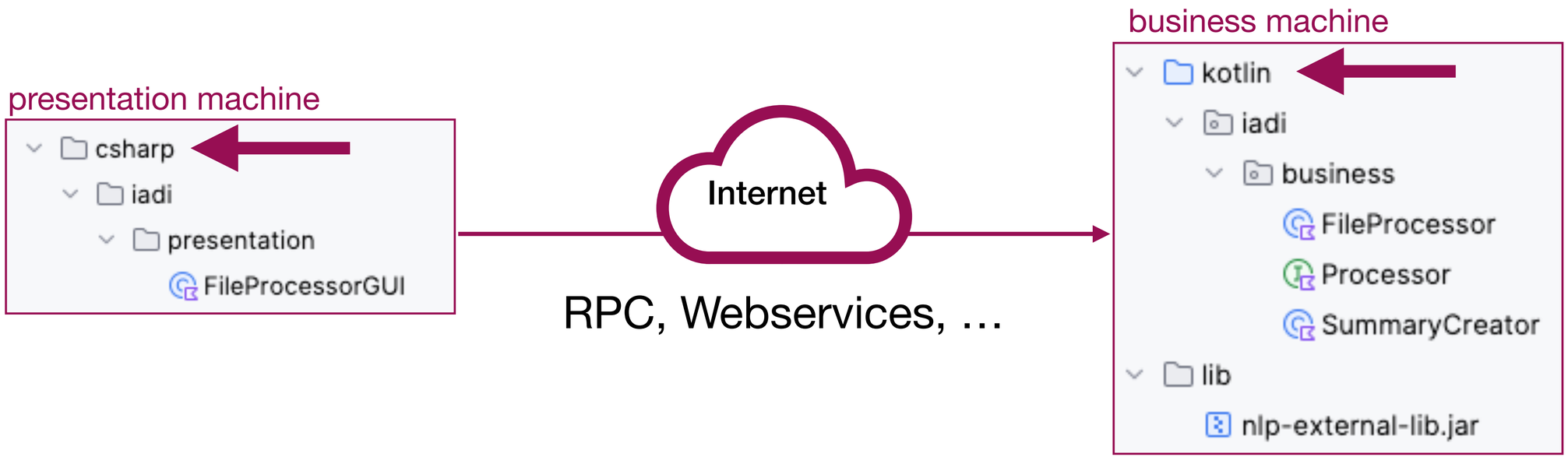
© Pedro Alves 2026

Modularization & Dependencies

Here comes Internet….

© Pedro Alves 2026

Modularization & Dependencies



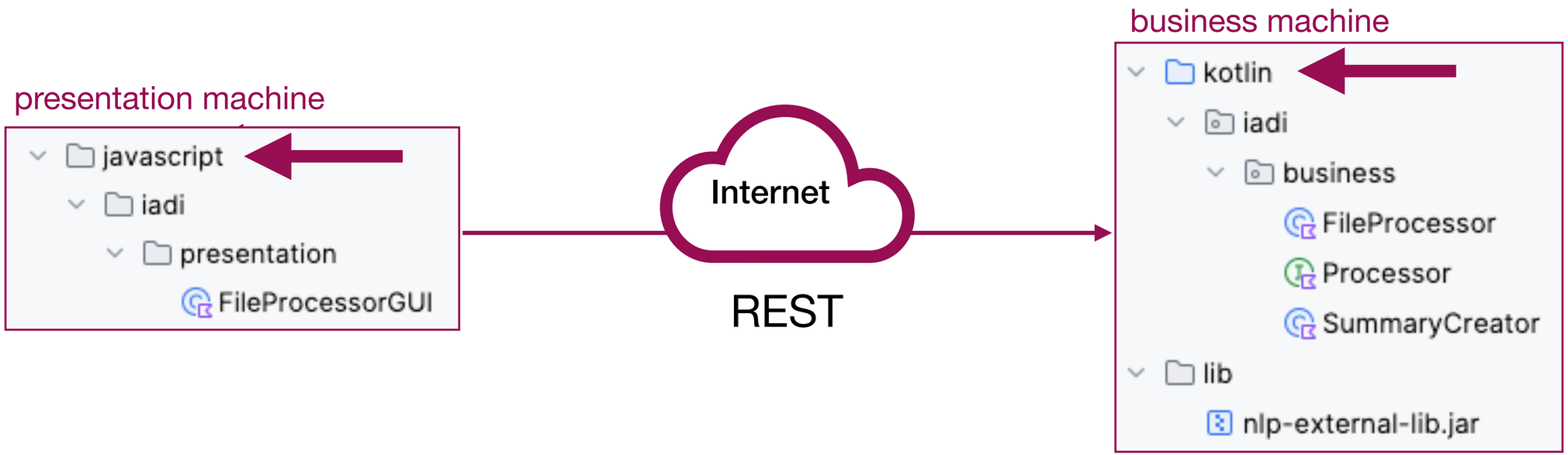
Client-server using native

- benefits: independent deployment, centralization of logic, easier updates (server-side), multiple clients supported
- problems: latency, network failures, security, not scalable to multiple domains

stronger decoupling through network boundaries

© Pedro Alves 2026

Modularization & Dependencies



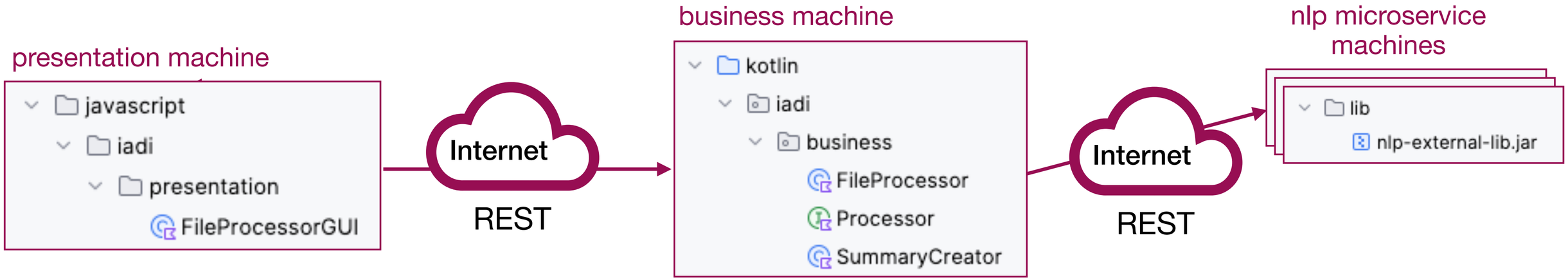
Client-server using the browser

- benefits: no need to install in the client, always up-to-date
- problems: language must be javascript(*), restricted environment, not a native UI

(*) Or a language that transpiles to javascript such as typescript

© Pedro Alves 2026

Modularization & Dependencies



Micro-services

- benefits: domain boundaries, team decentralization, deployment autonomy, horizontal scaling
- problems: the same of client-server + complexity

© Pedro Alves 2026

Detour - Scaling

Vertical scaling

- **Make one machine stronger (increase CPU, memory, …)**
- Benefits: simplest approach, no architectural changes, easier debugging, no replication logic
- Problems: hardware limits, expensive, single point of failure
- Mostly used in Client-Server

Horizontal scaling

- **Add more machines**
- Benefits: theoretically unlimited scaling, cheaper hardware, fault tolerance
- Problems: complexity, consistency and synchronization problems, operational overhead
- Mostly used in micro-services

© Pedro Alves 2026

Modularization & Dependencies

In summary

- Functions
- Classes
- Packages
- Layers
- Libraries (*)
- Separate processes
- Client-Server
- Micro-services

small modules

- ✅ Isolation
- ✅ Team distribution
- ✅ Deployment independence

- 👎 Increased complexity
- 👎 Network failures
- 👎 Higher need for team communication

large modules

(*) Includes maven multi-module

© Pedro Alves 2026

Exercise

	Benefit over previous
everything in main()	-
Functions	readability, reuse
Classes	
Packages	
Layers	
Libraries	
Separate processes	
Client-server	
Micro-services	

© Pedro Alves 2026

Resolution

	Benefit over previous
everything in main()	-
Functions	readability, reuse
Classes	encapsulate state + behavior
Packages	organize related classes
Layers	enforce dependency rules: unidirectional, ordered
Libraries	independent code-bases, team autonomy
Separate processes	different runtime/language, fault isolation
Client-server	independent deployment between client and server
Micro-services	independent deployment between server modules

© Pedro Alves 2026